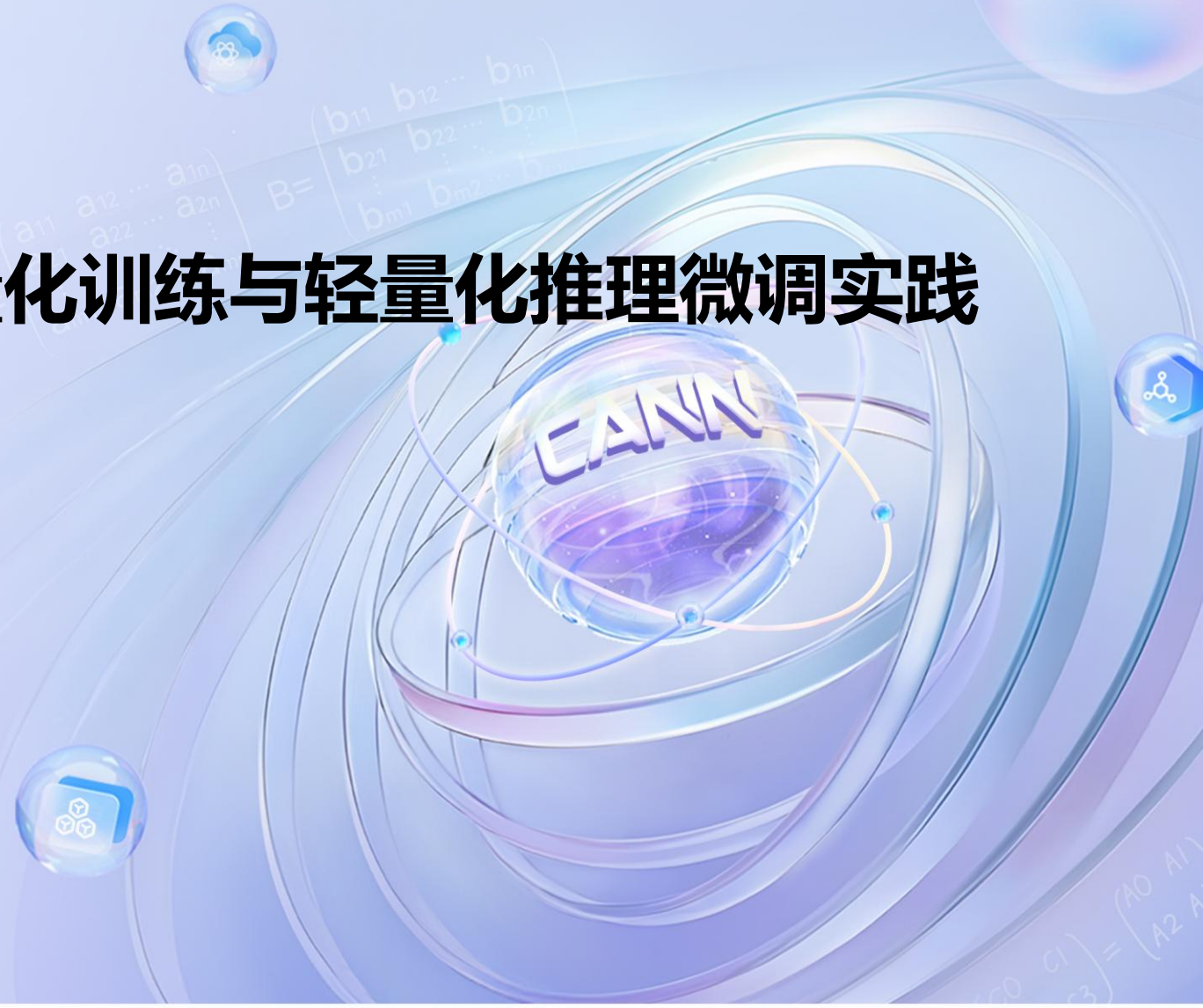
基于Ascend 950 低精量化训练与轻量化推理微调实践

时间：2026/10/20



CANN

目录

Part 1低精度量化训练: MXFP4/MXFP8与HiF8实践

Part 2 昇腾+鲲鹏异构单卡推理及单机LoRA实践

大模型低精度量化训练优化 通过低精实现显存容量、计算和通信开销的降低

低精训练的目标：在尽量保证精度的前提下，把 BF16 计算逐步降到 FP8 / FP4等更低精度，从而达成以下目的：

降低计算开销

FP8的矩阵计算，在常规尺寸下相较BF16带来成倍的加速

降低数据搬运和显存压力

低精通信可以缓解带宽压力，低精KV等模块可以缓解显存压力

统一低精能力

通过统一的训练流程框架，只需修改配置，即可切换预训练和后训练，提升训练易用性

真正的低精训练并不只是把 BF16 Tensor cast 成 FP4/8这么简单，需要解决如下三个问题

哪些模块需要量化?

哪些层的权重、激活、梯度、Compressed KV等需要量化，量化对象的范围需要明确

不同模块用哪种低精策略?

MXFP8 / Block FP8 / HiFloat8 / MXFP4 QAT / Q-DQ 伪量化 —— 同一模型里多种策略并存

怎么和现有训练栈结合?

需要协同TorchTitan-NPU的模型构建、FSDP、torch.compile，实现性能最优

CANN社区低精训练Recipe：以 TensorSwap + ModuleSwap 作为统一接入机制，模型代码零改动实现低精替换

https://gitcode.com/cann

CANN

Ascend 950 低比特能力汇总

CANN支持端到端低比特计算优化，覆盖模型训练与推理，加速开源大模型生态适配

50+	4	6
低比特算子	低bit浮点格式	算子大类

01 格式覆盖全面

支持 FP8 (E4M3/E5M2)、MXFP8 (E8M0)、HiFP8 (自研非 IEEE)、MXFP4 (E2M1 E8M0) 四大低比特数据类型。

02 算子覆盖广泛

低比特支持横跨 Attention、MoE、MatMul 、Norm、Conv、Quant、通信融合等 6 大类 50+ 算子

03 端到端低比特流水线

从量化 (DynamicMxQuant)
→ 计算 (QuantBatchMatmul)
→ 通信 (QuantMatmulAllReduce)
→ 反量化 (AscendAntiQuant，一般不单独调用，都是和其他算子融合) 全链路低比特

04 多种量化粒度

支持 per-token、per-group (32/64/128)、per-block、per-tensor、per-channel、MX 等多级量化粒度，适配不同精度-性能需求

05 融合量化算子

RmsNormQuant、SwigluMxQuant 、AddRmsNormDynamicMxQuant 等量化融合算子减少访存开销，提升端到端性能

06 通信量化一体

MC2 算子支持低比特通信量化，实现计算与通信的低比特闭环，最大化集群效率

CANN

如何使能低精训练 面向开源生态，继承上游torchao特性，使能Ascend 950低精训练

TorchTitan：先定义，再构建（决定模型如何创建）

Module.Config

结构 / 超参 / 子模块配置

build()

根据 Config 实例化

nn.Module

构建得到的模型实例

Linear.Config(in_features=4096, out_features=4096)

TorchAO：先有模型，再进行量化（决定已有模型如何量化）

nn.Module

已经构建好的模型

+

AOBaseConfig

量化类型/ 粒度 / 模块

quantize_()

量化后模型

对现有模块进行转换

Config工作流区别：TorchTitan 的 Config 用于构建模型；TorchAO 的 Config 用于配置已有模型的量化转换。

如何使能低精训练 面向开源生态，继承上游torchao特性，使能Ascend 950低精训练

融合方式：Build 前确定量化配置

量化策略和模型结构一起参与构建

Module.Config
模型结构

+

AOBaseConfig
量化策略

配置融合 / 子模块配置转换
从 Config 中选择低精度模块

生成低精度配置

Quantized Module.Config
目标模块的低精度构造描述

build()

低精度模型模块
无需构建后的模块结构替换

核心步骤：融合 Config → 直接构建低精度模块

核心优势

量化纳入构建生命周期

01 构建执行更统一
build() 统一触发量化，无需在外部单独编排

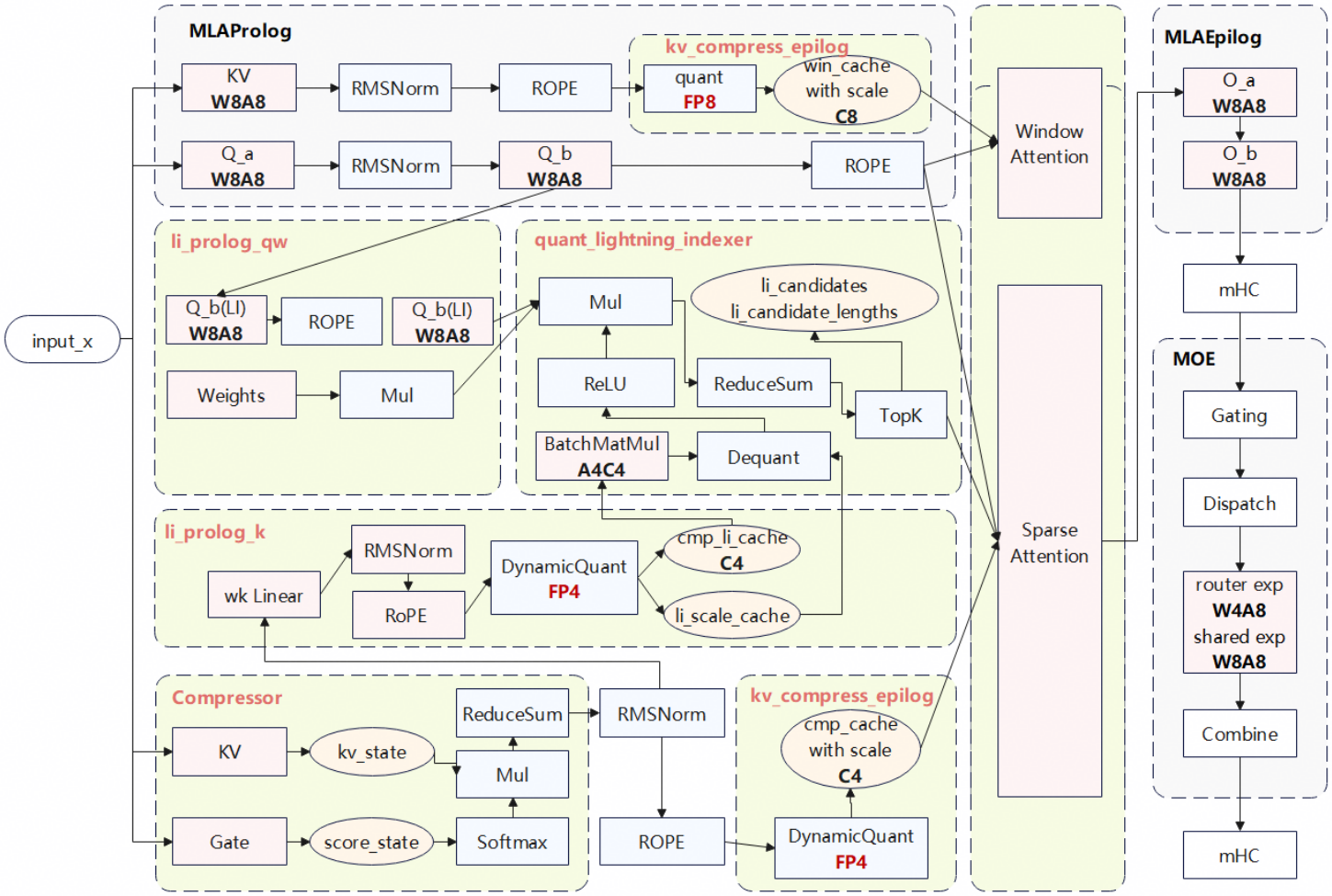
02 初始化开销可优化
配合低精度权重直建/加载，减少高精度中间态

03 更易与分布式协同
便于协调量化、模型并行与编译的执行时序

04 混合精度配置更灵活
按层或子模块组合 FP8、MXFP4/8、BF16 等策略

05 统一管理，扩展更容易
量化策略可校验、复现，也便于接入新量化方案

以DeepSeek V4.1为例，复刻原生低精量化方案 包含MM/GMM模块，以及LI，Compressor等特色模块



整体量化方案

MLAProlog:

- Q_a, Q_b, KV, 采用W8A8
- win_kv采用C8伪量化，保证训推一致性，量化粒度复刻原生

IndexerProlog与 LightningIndexer:

- Q_b, 采用W8A8，$weight_proj$保持BF16
- Li BMM采用A4C4

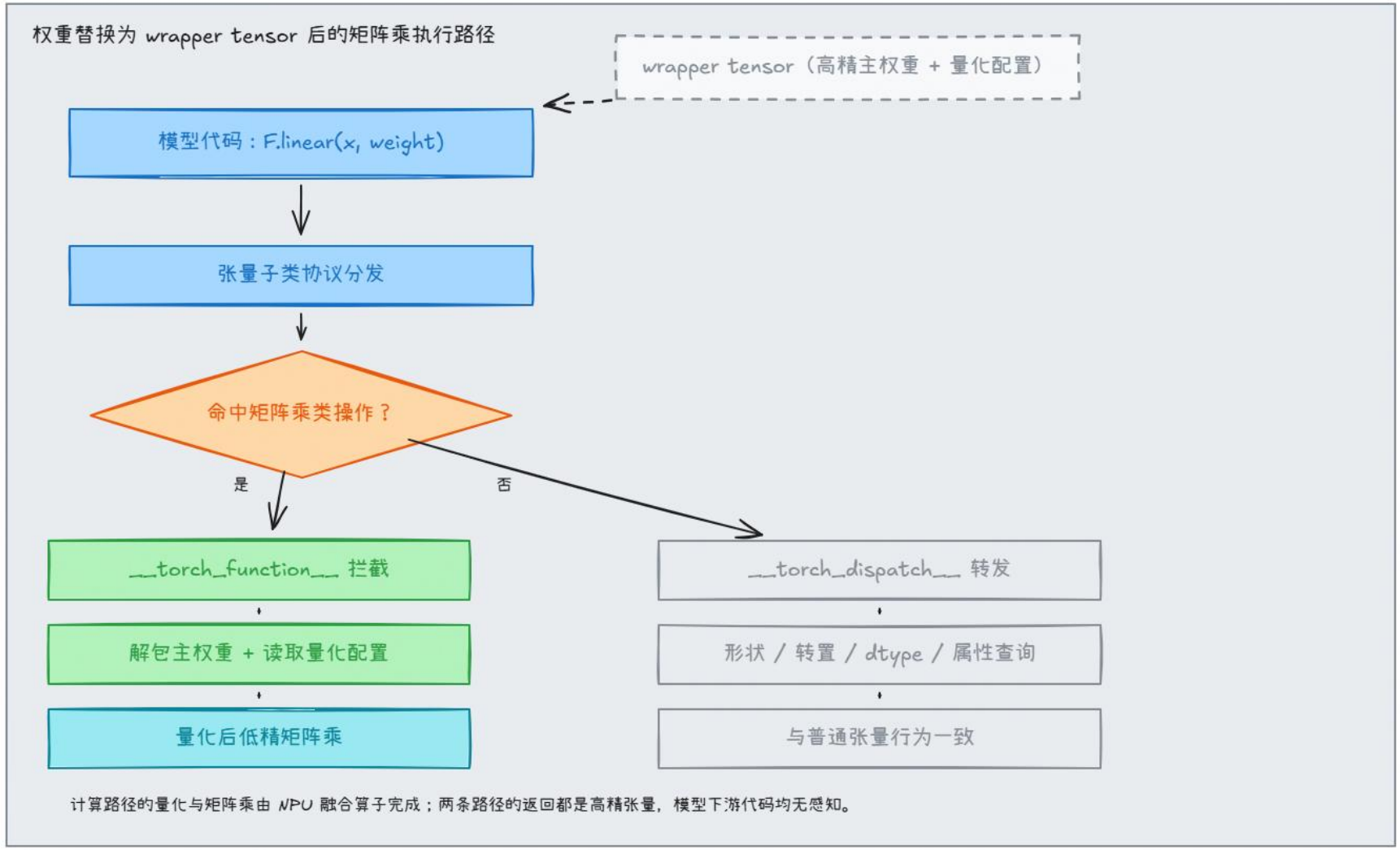
Compressor:

- $wkv, wgate$保持BF16，压缩过程中的计算均保持浮点路径
- com_kv使用C4伪量化，保证训推一致性，量化粒度复刻原生

W8A8表示权重使用Block FP8，激活使用MXFP8，同时基于Ascend 950自有的高性能HiFloat8格式，新增W8A8-HiF8的量化方案

如何使能低精计算 使能DeepSeek v4/4.1 预训练, SFT, RL；预训练: A8W8；QAT: A8W4

Tensor替换：Linear/BMM/GMM 的 weight 变成 Wrapper Tensor，运行期由 __torch_function__ 或 __torch_dispatch__ 拦截



量化算子MXFP/HiFP：

mx_to_block_mx_quant

npu_dynamic_quant

npu_dynamic_block_mx_quant

已有多数据格式多细粒度的高性能量化算子

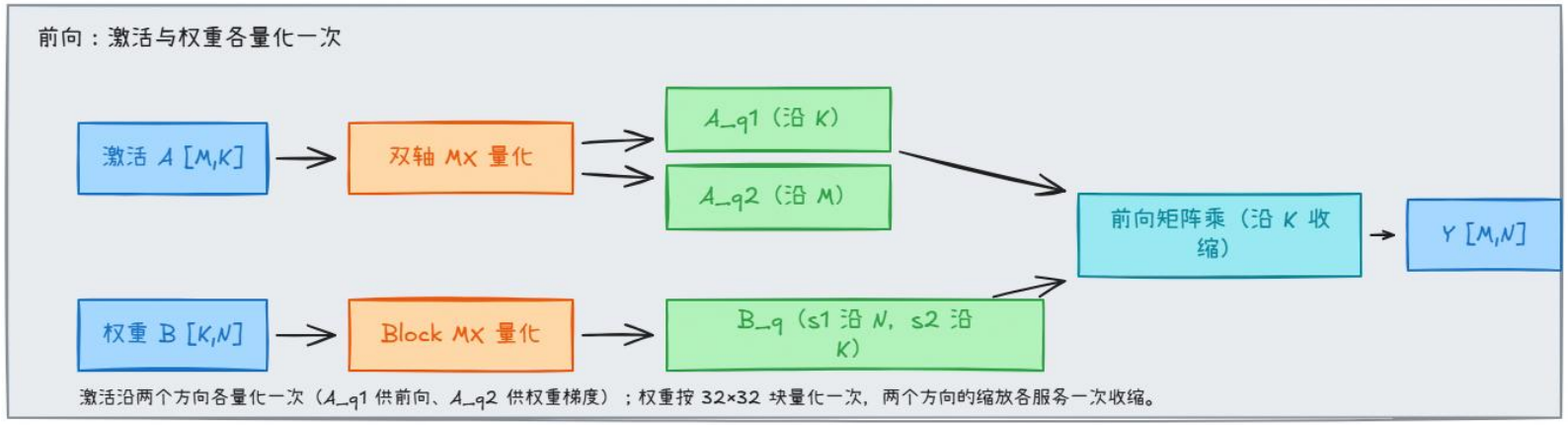
矩阵乘算子：

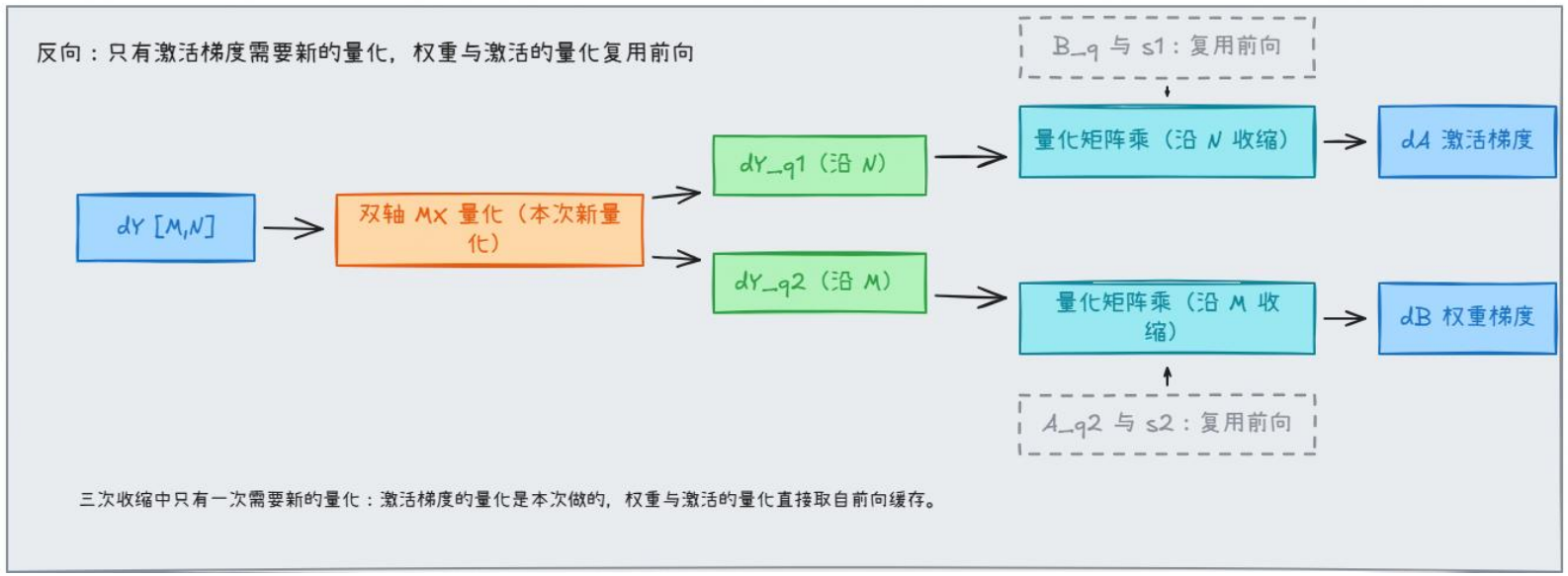
npu_quant_matmul

npu_grouped_matmul

如何使能低精计算 使能DeepSeek v4/4.1 预训练, SFT, RL；预训练: A8W8；QAT: A8W4

量化方案 – MXFP8：权重量化：Block MXFP8；激活量化：标准 MXFP8 的双轴实现





前反向计算

权重量化： Block MXFP8：每 32×32 块共享一个缩放系数，对权重进行量化，相比上一版128x128粒度更细，同时保证前反向数值一致性

激活量化： 标准MXFP8，使用**双轴量化算子**，同时出K轴和M轴的量化数据，利用融合算子减少量化开销

矩阵乘： 反向只需要一次新的量化。前反向三个矩阵乘中，只有激活梯度dY需要量化，用**双轴量化算子(***npu_dynamic_mx_quant_with_dual_axis***)**同时出M轴和N轴的量化数据

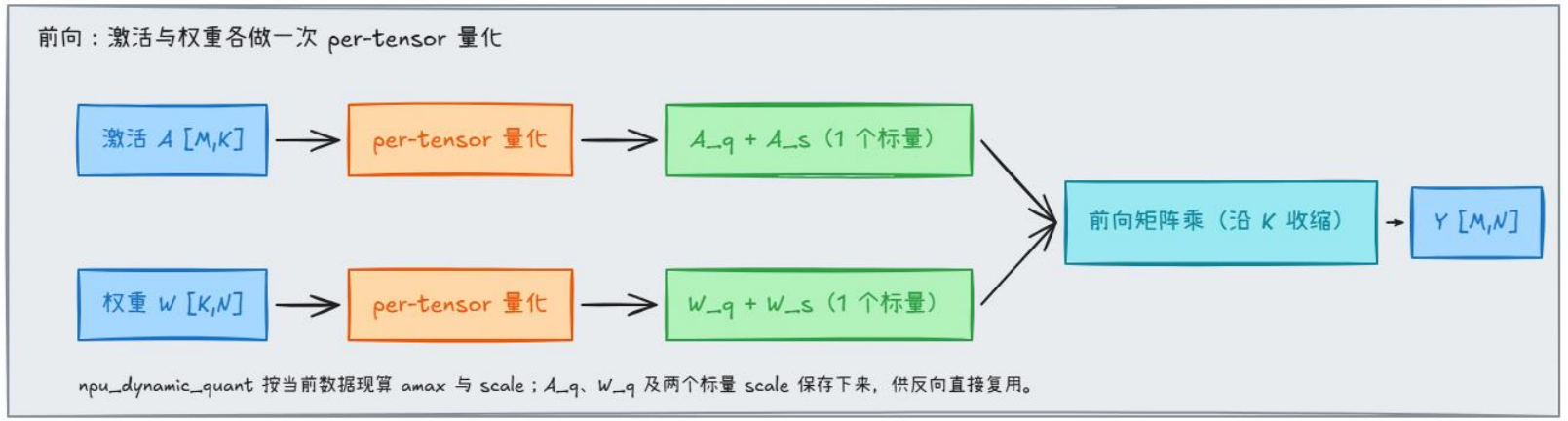
性能相比bf16提升1.2x

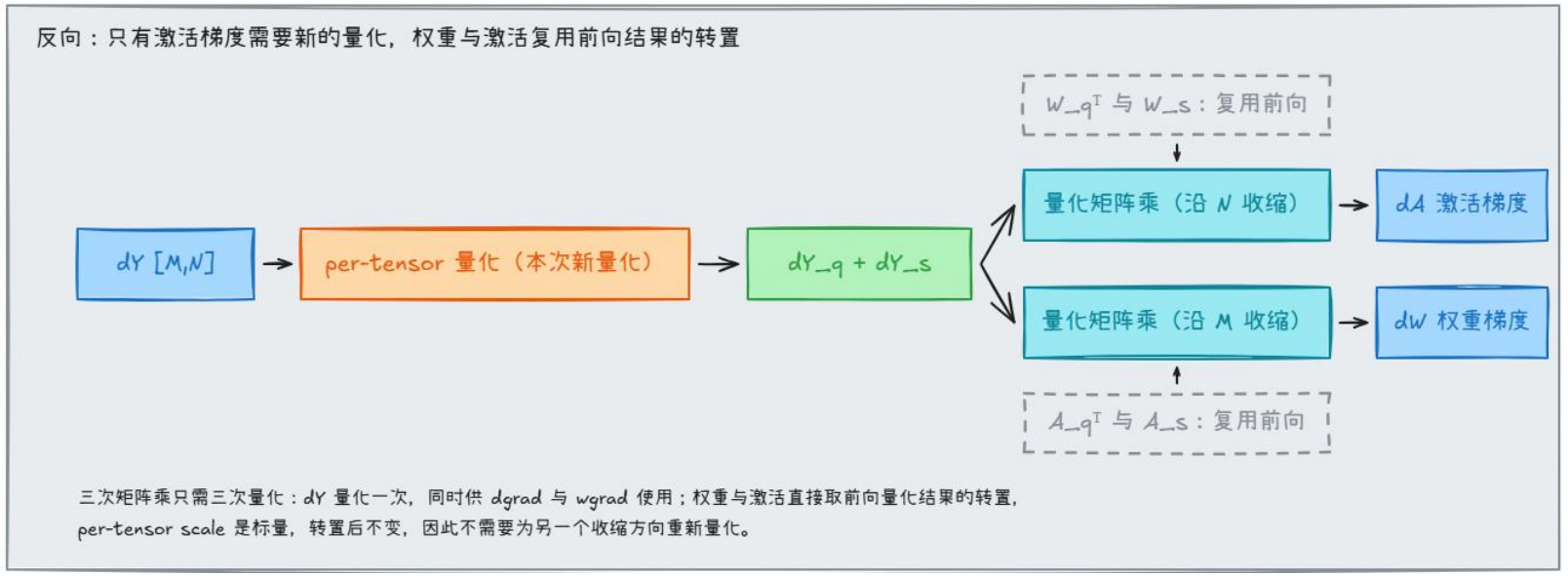
https://gitcode.com/cann

CANN

如何使能低精计算 使能DeepSeek v4/4.1 预训练, SFT, RL；预训练: A8W8；QAT: A8W4

量化方案 – HiFloat8：权重、激活与激活梯度都使用 HiF8 per-tensor 量化，scale 采用 current tensor scaling





前反向计算

前向对激活与权重各量化一次；

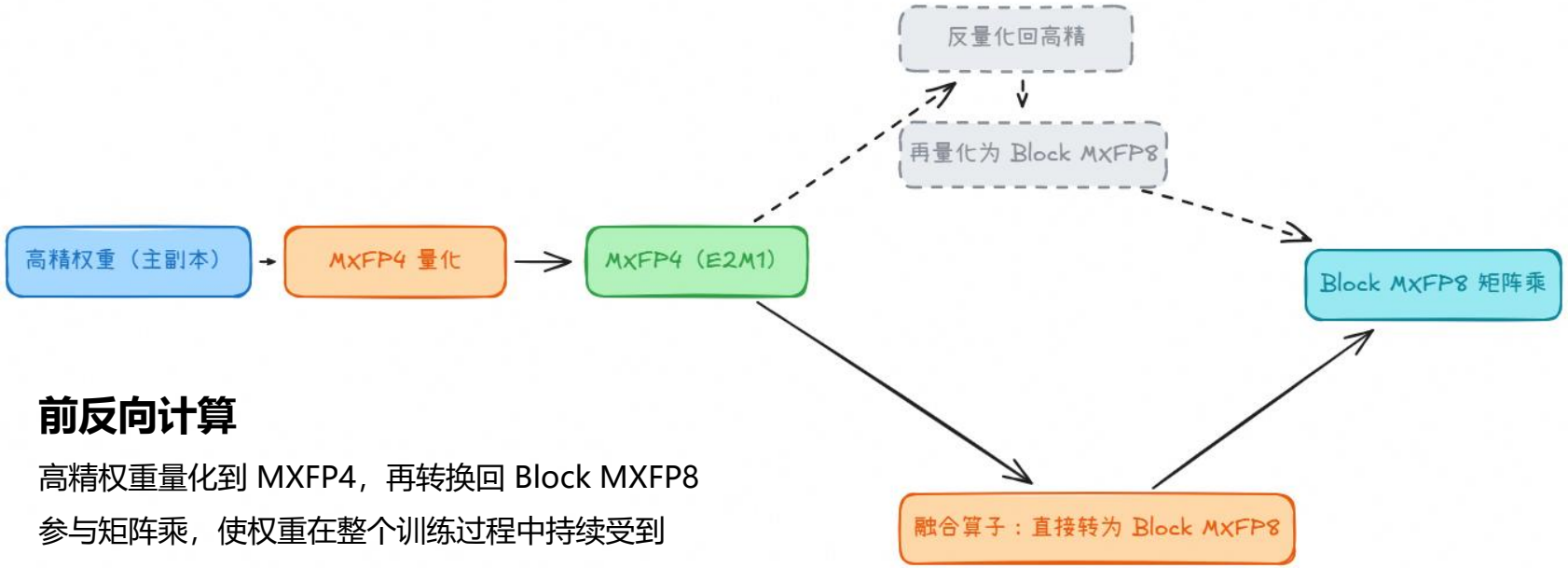
反向中只有激活梯度需要新的量化，权重与激活均复用前向量化结果的转置，per-tensor scale 在转置后不变，相比较mxfp类的per-group/block scale，省略了二次量化的过程，极大提升整网性能；

Scale优化：current scaling 每次量化前都要把整个张量读一遍求 amax，这是纯访存开销。后续可引入 delay-scaling：连续若干步复用同一个 scale，只在窗口末尾更新，以省掉大部分 amax 统计

性能相比bf16提升1.3x

如何使能低精计算 使能DeepSeek v4/4.1 预训练, SFT, RL；预训练: A8W8；QAT: A8W4

量化方案 – QAT(MXFP4)： 路由专家权重在训练完成后可以直接以 MXFP4 形式部署



前反向计算

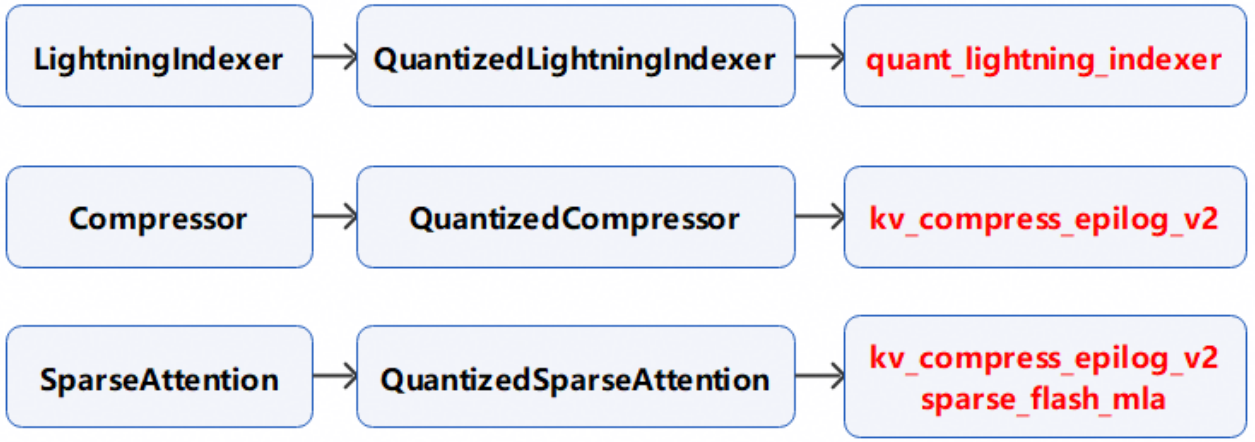
高精权重量化到 MXFP4，再转换回 Block MXFP8
参与矩阵乘，使权重在整个训练过程中持续受到
MXFP4 数值格点的约束

性能优化：朴素做法是先把 MXFP4 反量化到高精，再重新量化为 Block MXFP8，经过两次量化，使用高性能融合算子(*mx_to_block_mx_quant*)一步完成MXFP4到Block MXFP8的转换，省去一次中间格式的往返
引入了MXFP4的量化，但是融合算子的优化保证性能与原生MXFP8性能持平

性能相比bf16提升1.2x

如何使能低精计算 高效低精融合算子替换，使能DeepSeek v4/4.1 预训练, SFT, RL；

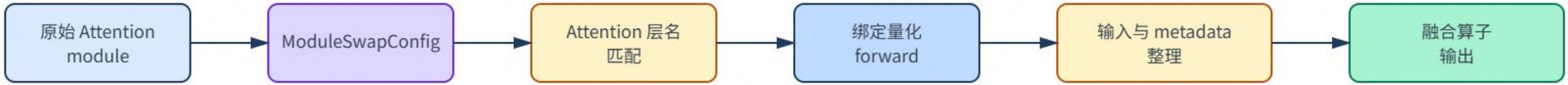
Module替换：以DeepSeek-V4.1融合算子接入为例，涵盖Lightning Indexer (LI)、Sliding-Window Attention KV (SWA KV)和Compressed KV (CMP KV)



使能**DeepSeek v4/4.1 Li算子**，支持原生A8C8-MXFP8，A4C4-MXFP4扩展版 A8C8-HiF8

使能**DeepSeek v4/4.1 compress kv, win kv C4/C8伪量化**，确保RL训推一致性

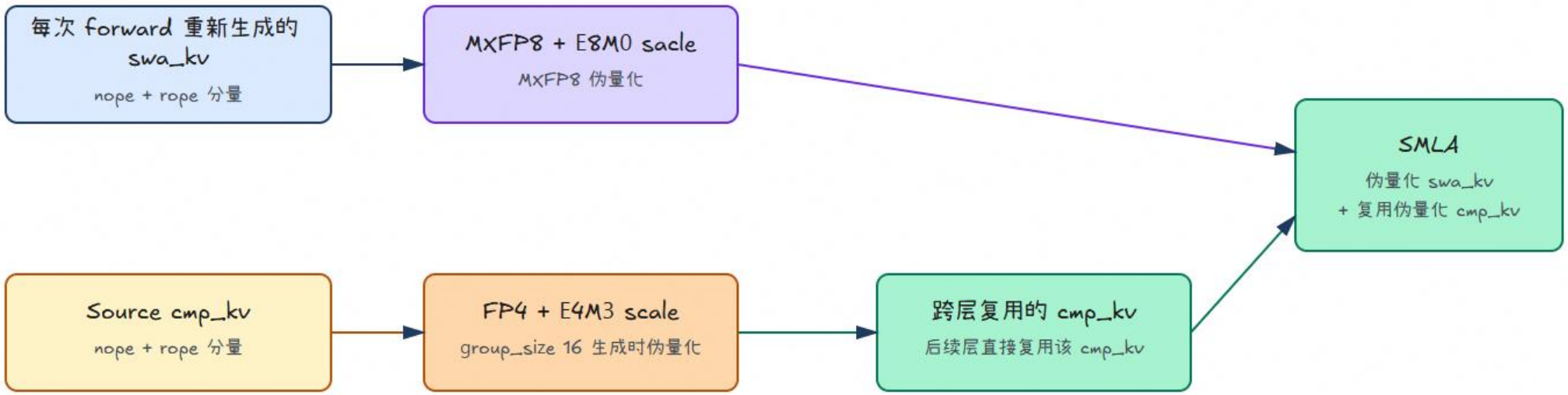
以Attention为例，融合算子为双激活输入，同时需要处理序列的CP切分、融合 kernel 的反向计算，通过替换module的forward行为使能低精



CANN

如何使能低精计算 高效低精融合算子替换，使能DeepSeek v4/4.1 预训练, SFT, RL

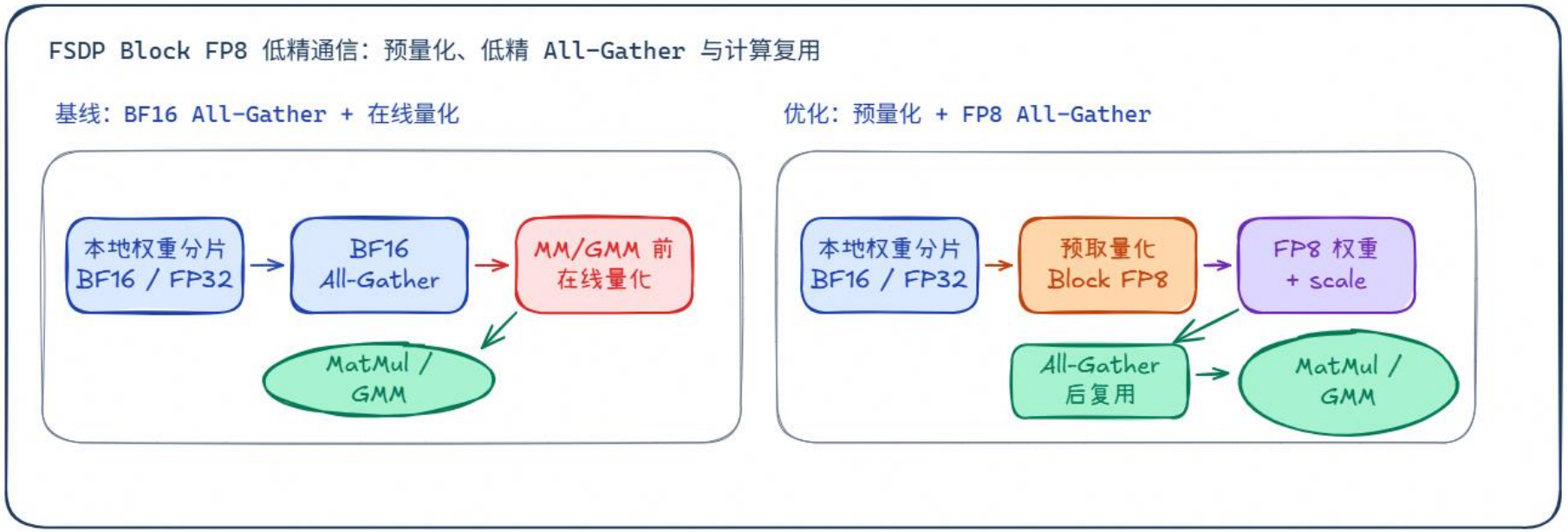
DeepSeek-V4.1：分离的 SWA KV 和 Compressed KV 量化



SWA KV：由当前 Attention 层在每次 forward 中重新生成，访问频繁且对量化误差敏感。使用 C8量化方式，按 1×32 粒度分组，在降低缓存带宽的同时保留较高数值精度，适合这条对输出敏感的路径

CMP KV：是经过压缩得到的 KV，单次 Attention 只访问候选子集，且同一份 compressed KV 可由多个 layer 复用。由于其缓存规模和跨层复用特征，存储效率和带宽占用更重要，因此采用 C4量化方式，按 1×16 粒度分组，在降低缓存容量的同时保留块级幅值信息

低精通信： FSDP Block FP8 低精通信，将量化边界前移



fsdp_pre_all_gather()

本地 Block-MX / FP8 量化

fsdp all-gather

通信：低精 data + scale

fsdp_post_all_gather()

重建完整低精 wrapper

- fsdp_pre_all_gather() 通信前将本地 shard 量化为 fp8 data+scale；
- FSDP all-gather 的传输数据，从 BF16 量化至 FP8 权重 + Block scale，减少参数通信量，并避免前向和反向重复量化权重
- fsdp_post_all_gather() 创建或更新 wrapper，内部低精 payload 可以直接供后续量化 GEMM 使用。

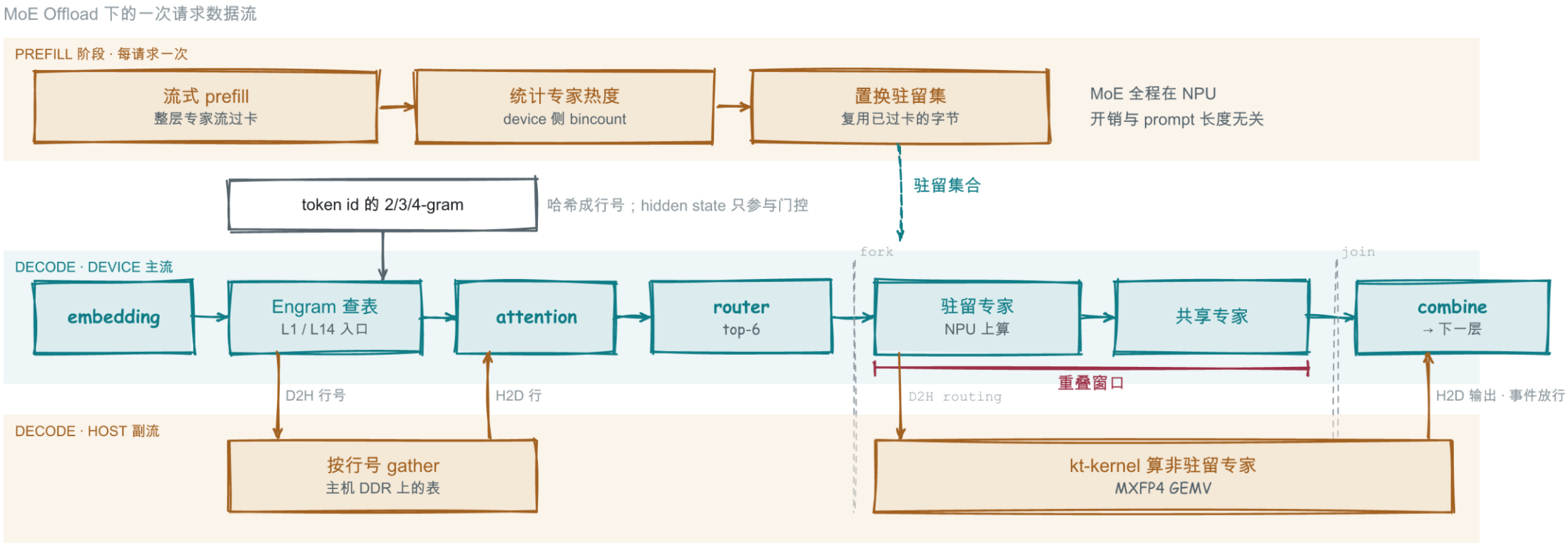
目录

Part 1低精度量化训练: MXFP4/MXFP8与HiF8实践

Part 2 昇腾+鲲鹏异构单卡推理及单机LoRA实践

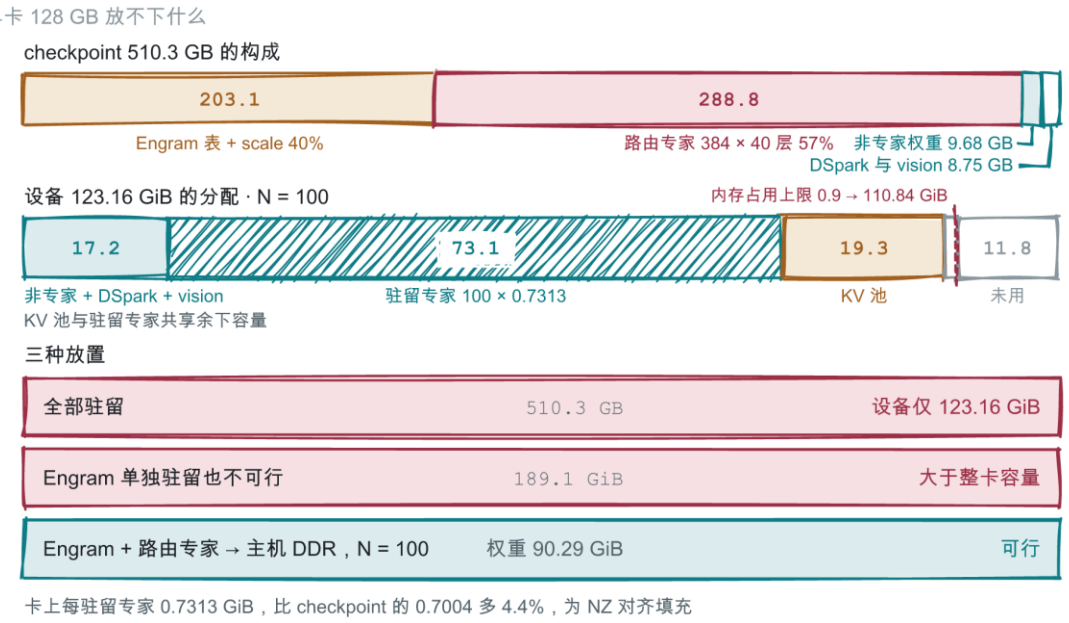
方案总览 MoE Offload 下的一次请求数据流

prefill 阶段标定驻留集；decode 阶段 device 主流与 host 副流并行，两类权重各走各的卸载路径。



单卡容量约束与卸载划分 checkpoint 510.29 GB 的构成与卸载边界

卸载判据有两项：占用容量，与每 token 必须读取的字节数。两者相差数个数量级，因此可按冷热画像划分。



部分	大小	占比	每 TOKEN 读取
Engram 表 + scale	203.07 GB	39.8%	**12.38 KB**
路由专家 MXFP4	288.78 GB	56.6%	**top-6 未命中部分**
attention / 共享 / embed	9.68 GB	1.9%	**全部**
DSpark + vision	8.75 GB	1.7%	**视启用**

冷热画像 每 GB 存储产生的每 token 流量：attention 1000 MB/GB、路由专家 15.2 MB/GB、Engram 0.061 KB/GB

两种稀疏性 MoE 每 token 只激活 top-6；Engram 每层只读表中 24 行

划分依据 权重总量与每 token 读取的字节数解耦，后者决定 decode 时延

416.7 GB
外置至主机 DDR，Engram 203.07 + 非驻留专家 213.58

90.29 GiB
die 侧权重：驻留专家 73.13 + 非专家 17.16

32.87 GiB
余量：KV 池、workspace 与未分配。N 取整 100 之后剩下的

DSpark 为 DeepSeek 公开的投机解码框架（arXiv:2607.05147），视是否启用计入容量

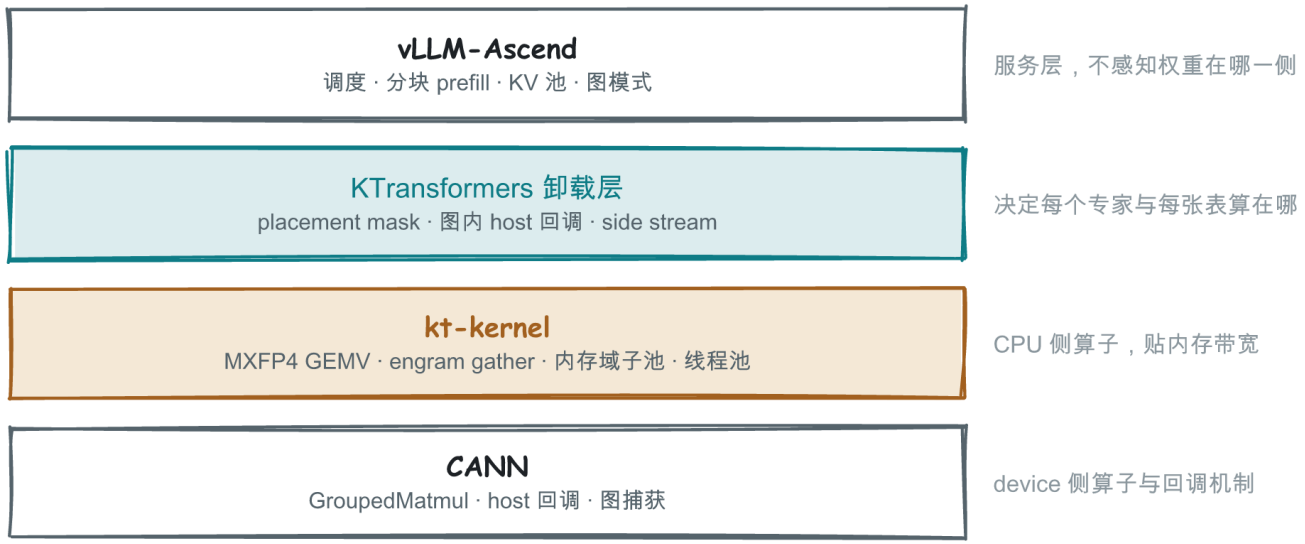
https://atomgit.com/cann

CANN

软件栈与两类权重的卸载路径分 卸载策略集中于 KTransformers 层，上下游保持社区实现

vLLM-Ascend 不感知专家权重所在侧；CANN 提供 host 函数节点与外部事件两项机制。

软件分层



四层自顶向下，卸载策略集中在第二层；上下两层保持社区实现

vLLM-Ascend 调度、分块 prefill、KV 池、图模式执行

KTransformers 卸载层 placement mask、副流 fork/join、外部事件放行

kt-kernel CPU 侧 MXFP4 GEMV、Engram gather、内存域子池与线程池

CANN GroupedMatmul、可投递的 host 函数节点、可由主机线程放行的外部事件

Engram 的一步：device 算行号 → D2H → host 按行号 gather → H2D → device 反量化；每 token 读 12.38 KB，是 checkpoint 里最冷的一类

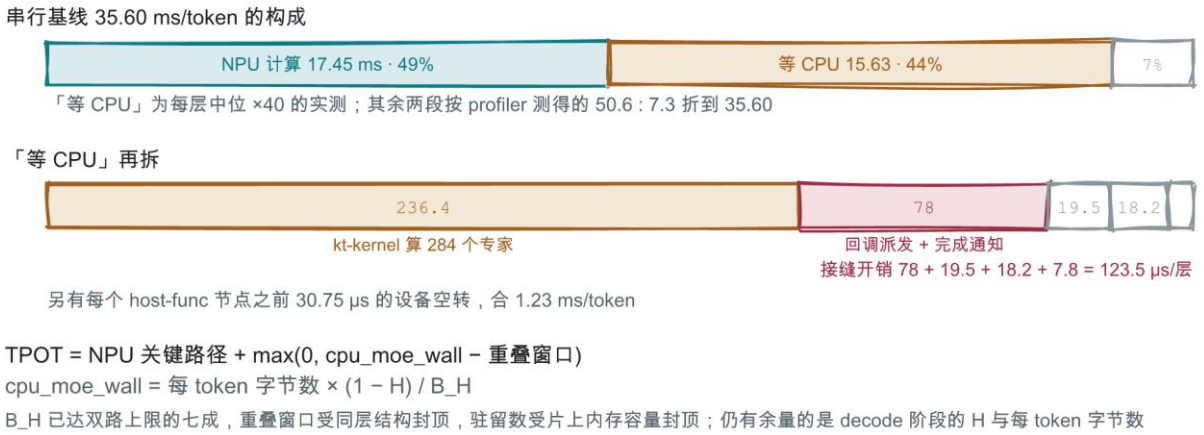
https://atomgit.com/cann

CANN

串行基线的时间构成 35.60 ms/token：四成多在等 CPU，其中六成是 CPU MoE 计算

基线分成 NPU 计算、等待 CPU、host 下发三段；「等 CPU」这一段再按每层中位逐项拆开。

每 token 时延的两级拆解



TPOT = NPU 关键路径 + max(0, cpu_moe_wall − 重叠窗口)

cpu_moe_wall = 每 token 字节数 × (1 − H) / B_H

B_H 已达双路上限的七成，重叠窗口受同层结构封顶，驻留数受片上内存容量封顶；仍有余量的是 decode 阶段的 H 与每 token 字节数

「等 CPU」的分段	µs/层	ms/token
kt-kernel 算 284 个专家	236.4	**9.46**
回调派发 + 完成通知	78.0	**3.12**
kt-kernel 的 Python 前奏	19.5	**0.78**
worker 唤醒 + 归并写回	18.2	**0.73**
卸载层自身的 Python	7.8	**0.31**
host-func 前的设备空转	30.8	**1.23**
合计	390.7	**15.63**

两个 GEMM 占 kt-kernel 计算的 92.3%；回调派发由 ACL 的单条报告线程串行承担，进程内所有流共用

9.46 ms
CPU MoE 计算，占「等 CPU」的 60.5%

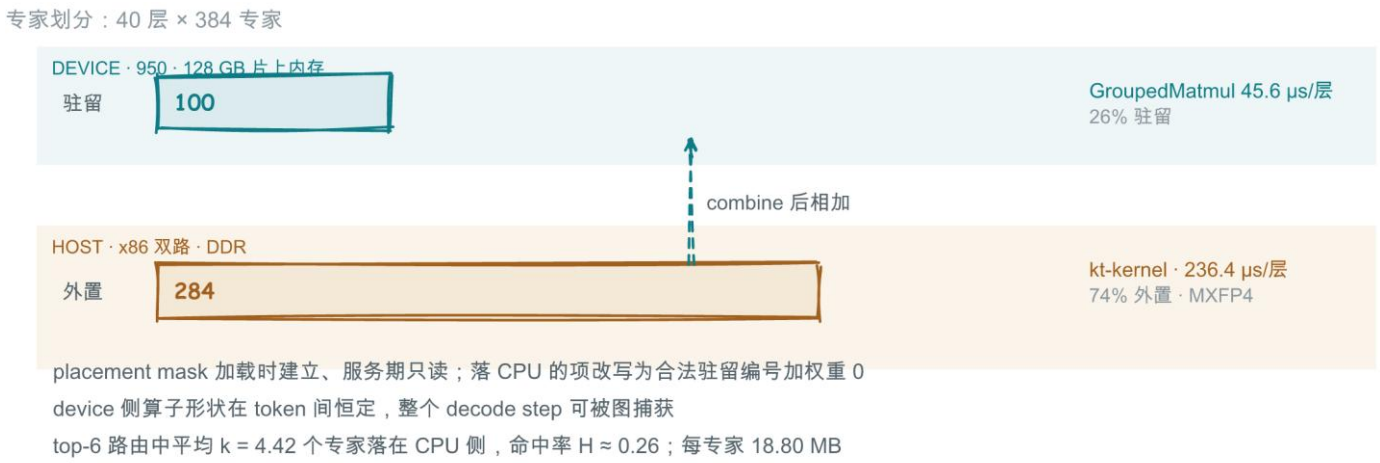
4.94 ms
接缝开销 123.5 µs/层，与专家数无关

1.23 ms
host-func 节点之前的设备空转

「等 CPU」15.63 ms 是每层中位 ×40 的实测，上表即它的分项；NPU 计算与 host 下发按 profiler 测得的 50.6 : 7.3 折到 35.60。单次测量有波动，占比只取整数位

MoE 专家的 CPU 卸载 284 / 384 个专家下放主机，device 侧算子形状恒定

卸载的约束条件是 decode step 仍可被图捕获，因此 device 侧算子形状须在 token 间保持不变。



topk_ids 中落 CPU 的项 := 任一合法驻留编号，对应 weight := 0

device 侧照常执行这些槽位，输出乘 0 不贡献结果；算子形状因此与 token 无关

placement mask 模型加载时建立、服务期只读；非驻留专家跳过 device 侧分配，权重不进 die

哨兵值的替代 昇腾路由算子不接受「不在本 device 计算」的哨兵值，落 CPU 的项改写为合法驻留编号加权重 0，device 侧照常执行且不贡献输出

形状恒定 device 侧算子形状在 token 间不变，整个 decode step 可被图捕获

路由统计 top-6 中平均 4.42 个落在 CPU 侧，命中率 H ≈ 0.26，等于驻留比例 100/384

100 / 384
每层驻留 NPU 的专家数

18.80 MB
每个专家的 MXFP4 权重

4.42 个
每层平均落在 CPU 的专家数

CPU 侧的带宽上限 实测等效带宽 352 GB/s，为本机双路上限的 71%

CPU 段耗时正比于从 DDR 读取的字节数。按字节数与实测时间折算等效带宽，可判断剩余余量。

CPU 侧 MoE 的带宽账

每层 CPU 耗时 = 固定头开销 + k × 每专家字节数 / B_H

k 落在 CPU 的专家数	实测，命中率 H ≈ 0.26	4.42 个/层
每专家 MXFP4 字节	3 × 5120 × 2304 × 0.53125 B	**18.80 MB**
每层要从 DDR 读	4.42 × 18.80 MB	**83.1 MB**
实测 kt-kernel 计算	profiler，两个 GEMM 占 92.3%	236.4 μs/层

83.1 MB / 236.4 μs = 352 GB/s



0.53125 = 4 bit 权重的 0.5 加 1/32 的 e8m0 scale；每专家三个矩阵

带宽已经吃到七成，这条路继续压的空间不大；真正能动的是 k —— 它由命中率决定

任务按需领取 任务粒度为(专家, 切片)对，由共享原子计数器分配，避免冷专家拖尾

绑核与分域 子池数等于内存域数，避免前若干 worker 集中在同一个 L3 域

余量在命中率 带宽已吃到七成，继续压的空间不大；真正能动的是 k

83.1 MB
每层要从 DDR 读的专家权重

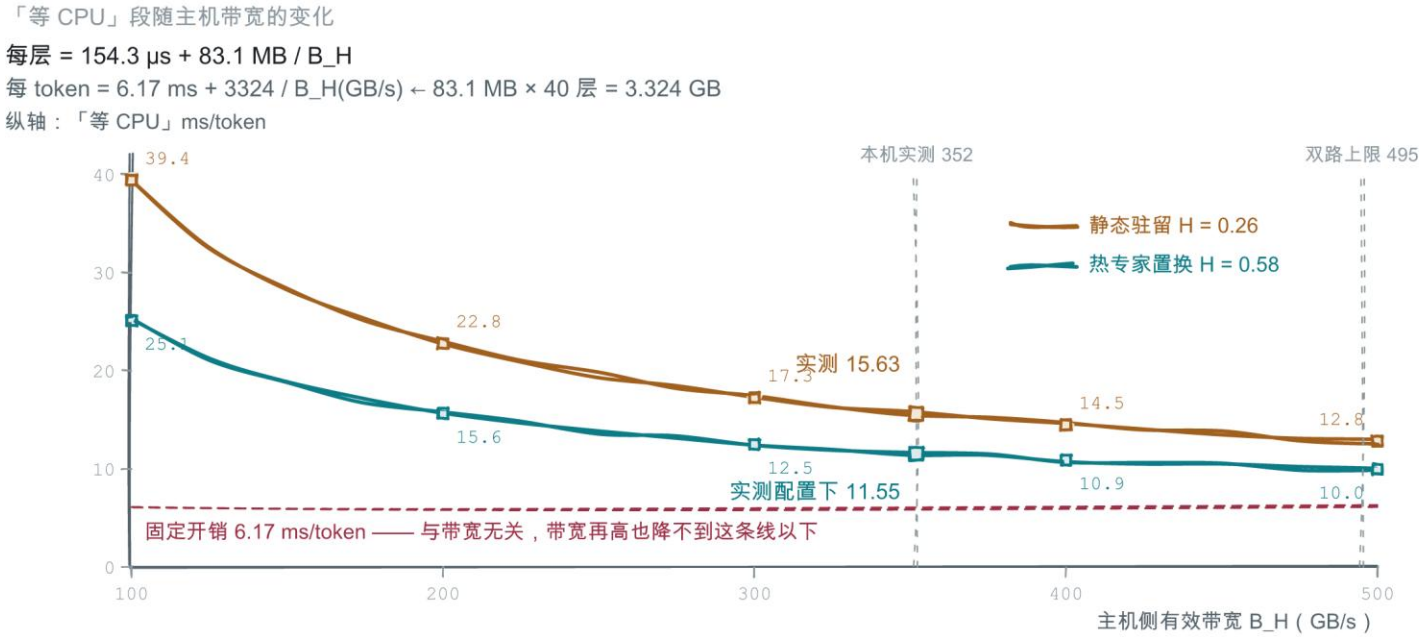
92.3%
CPU 段里两个 GEMM 的占比

9.46 ms
CPU MoE 计算，236.4 μs/层 × 40

CANN

带宽推演 100 - 500 GB/s 下「等 CPU」段的推演

把实测的 236.4 μs/层 还原成公式里的带宽项，就能推演其它带宽下的「等 CPU」段。



带宽只缩放 kt-kernel 的 GEMM 那一项；命中率缩放的是同一项的系数 —— 两条曲线形状相同，只差 k

模型对得上 352 GB/s 代入得 15.61 ms，实测 15.63，差 0.1%

地板 6.17 ms 每层 154.3 μs 的固定开销与带宽无关；带宽再高也停在这条线上

两种改法的量级 带宽 352 → 704 GB/s，15.61 → 10.89；同一台机器上命中率 0.26 → 0.58，15.61 → 11.55

同一项的两个因子 带宽和命中率都作用在 k × 每专家字节数 / B_H 上，所以两条曲线形状相同、只差系数

15.61 ms
模型在 352 GB/s 的预测，实测 15.63

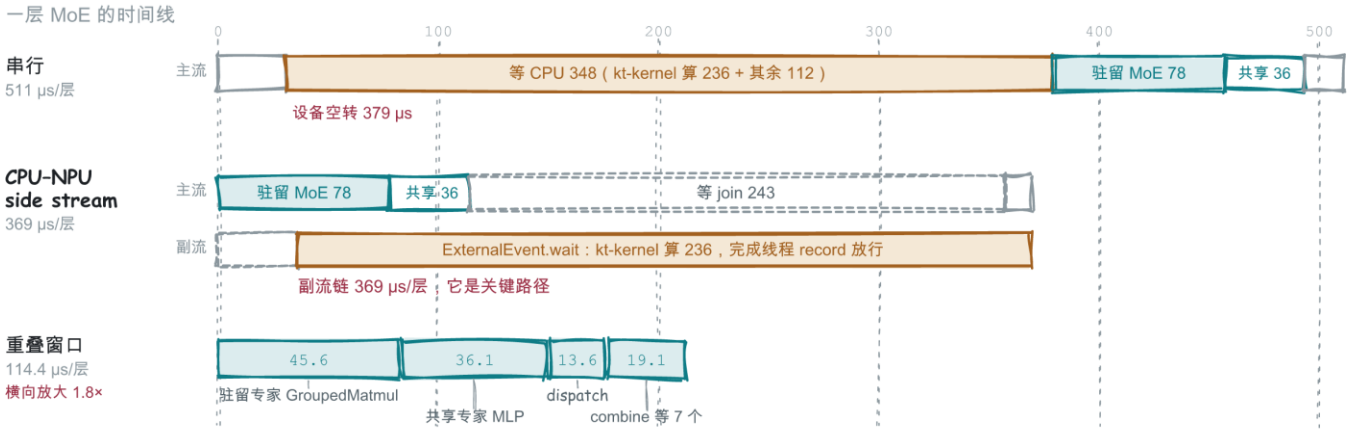
6.17 ms
与带宽无关的固定地板

11.55 ms
同带宽下 H = 0.58 的预测

CANN

device 与 host 的并行 host 路径移出主流，35.60 → 30.31 ms/token

串行实现下每层 MoE 段 511 µs，其中约 379 µs 为设备空转。三项改动依次把该区间用于主流计算。



驻留 MoE = GroupedMatmul 45.6 + dispatch 13.6 + combine 等 19.1 = 78.3 µs/层，加共享专家 36.1 为 114.4

每层 511 → 369 µs；端到端 35.60 → 30.31 ms/token，约 −5.3

窗口右边界由结构给定：合并之后是下一层 attention，跨层会改变数值

改动	窗口内容	ms/token
副流重叠	驻留 MoE 78 µs/层	**-3.00**
外部事件等待	缩短完成通知 13 µs/层	**-0.53**
共享专家进窗口	窗口 78 → 114.4 µs	**-1.71**

fork / join 的落点 fork 记在本层 routing 与三次 device fill 之后，dispatch 因此落入窗口；join 记在 NPU combine 之后

外部事件等待 host-func 节点只提交立即返回，设备停在 ExternalEvent.wait 上，由完成线程在 cpu_infer.sync 返回后 record 放行

共享专家 其输入即 MoE 层的输入，与 CPU 结果无数据依赖，只改调用顺序即可提前

511 → 369 µs
每层 MoE 段的实际耗时

379 µs
串行实现下每层的设备空转

42 → 2 次
每步主流被 host 阻塞的次数

收益的结构上限 重叠窗口 114.4 μs/层，由同层结构封顶

fork 与 join 之间主流可执行的算子即重叠窗口，它封顶副流重叠与共享专家两项的收益。

窗口内的算子	μs/层	ms/token
GroupedMatmul（驻留专家）	45.6	**1.82**
共享专家 MLP	36.1	**1.44**
MoeInitRoutingV3（dispatch）	13.6	**0.54**
Cumsum / MoeFinalizeRoutingV2 等	12.2	**0.48**
Index / Cast / Mul，卸载自身引入	6.9	**0.28**
合计	114.4	**4.58**

Index / Cast / Mul 把逻辑专家号换成 device 侧槽位号，并把下放专家的门控权重清零；这部分开销落在窗口内，被 CPU 计算完全覆盖。

左边界 CPU 要用 routing 的结果，fork 只能在 routing 之后

右边界 合并之后即下一层 attention；把 CPU 贡献推迟到下一层会改变数值

关键路径 副流链 369 μs/层（D2H 2.5 + 调起前空隙 35.75 + NOTIFY_WAIT 329.6），大于窗口，所以它才是关键路径

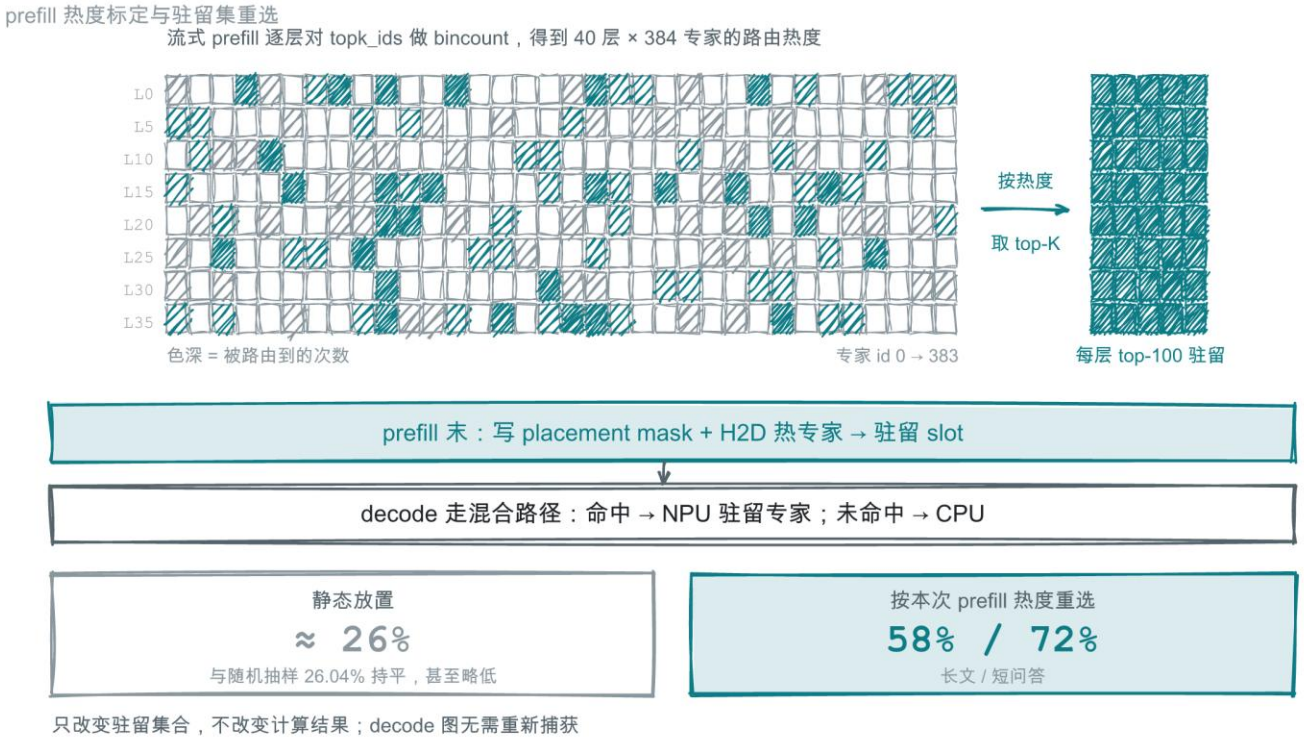
两种口径 本表是窗口内算子的执行时间合计；第 9 页的 -3.00 与 -1.71 是端到端配对实测，合计 4.71 ms，与窗口内的算子时间4.58ms基本能对上

端到端对账 三项 -3.00 / -0.53 / -1.71 合计 -5.24，对端到端实测的 -5.29

CANN

热专家标定置换 覆盖率 25.2% → 58%，TPOT 29.77 → 25.29 ms/token

同一请求内，prefill 激活的专家与 decode 命中的专家相关性高；静态放置不利用该相关性，其覆盖率等于驻留比例。



标定 prefill 期 device 侧 bincount 统计每层激活，热度累计自整个 prefill

置换 驻留集合在最后一个 chunk 上一次性换入，路由结构原地改写，decode 图无需重捕获

换入开销 复用流式 prefill 已搬上卡的字节，每请求换入 143.4 MiB、耗时 4.7 ms

流式 prefill 每次前向的开销固定，与 prompt 长度无关：prompt 增至 4 倍时 TTFT 仅增 8%

-4.48 ms
TPOT 29.77 → 25.29，约 15%

143.4 MiB
每请求换入的字节，耗时 4.7 ms

0.56 ×
覆盖率 25% → 58% 后 CPU 侧工作量的比值

结果、精度与开源复现 单卡 25.29 ms/token，精度与多卡基线持平

三组数据集上单卡与多卡基线的差值落在波动范围内。卸载改变权重的存放位置，不改变计算内容。

单卡 decode 时延，bs=1



数据集	题数	单卡 950	多卡基线
GSM8K	1319	97.27%	**97.12%**
Math500	500	97.00%	**97.40%**
GPQA-Diamond（non-thinking）	198	76.26%	**76.26%**

仓库与路径 atomgit.com/cann/cann-recipes-infer，样例在 integration/vllm/dsv41-flash-single-npu-moe-offload

五步脚本 01-config 配置 → 02-fetch 取码并打 10 个 patch → 03-build 编译 → 04-engram 排布 /dev/shm → 05-serve 拉起

关键参数 RESIDENT 每层常驻专家数（默认 100）、PROFILE lowlatency / throughput、MAXLEN、MAXSEQS

环境要求 1× 昇腾 950 + x86 AVX512-BF16；主机内存 ≥ 560 GiB，/dev/shm ≥ 200 GiB；CANN 9.2.0-beta.2

三棵源码树按 40 位提交号固定，patch 应用后 git write-tree 逐位校验 · 设计文档 docs/models/deepseek_v4_1/dsv41-flash-950-single-card.md

https://atomgit.com/cann

CANN

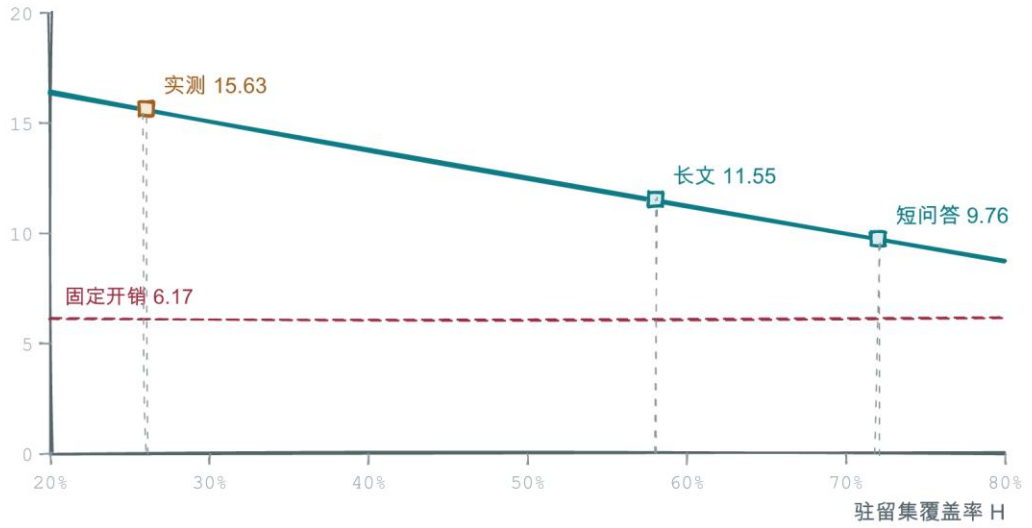
下一步 两个待验证的方向：提高命中率与降低位宽

两者作用在同一项上 —— k × 每专家字节数 / B_H。下面两张是按实测参数做的推演，尚无实测。

命中率 vs「等 CPU」· 推演

B_H = 352 GB/s 固定；t = 6.17 + 12.82 × (1 − H)

纵轴：「等 CPU」ms/token



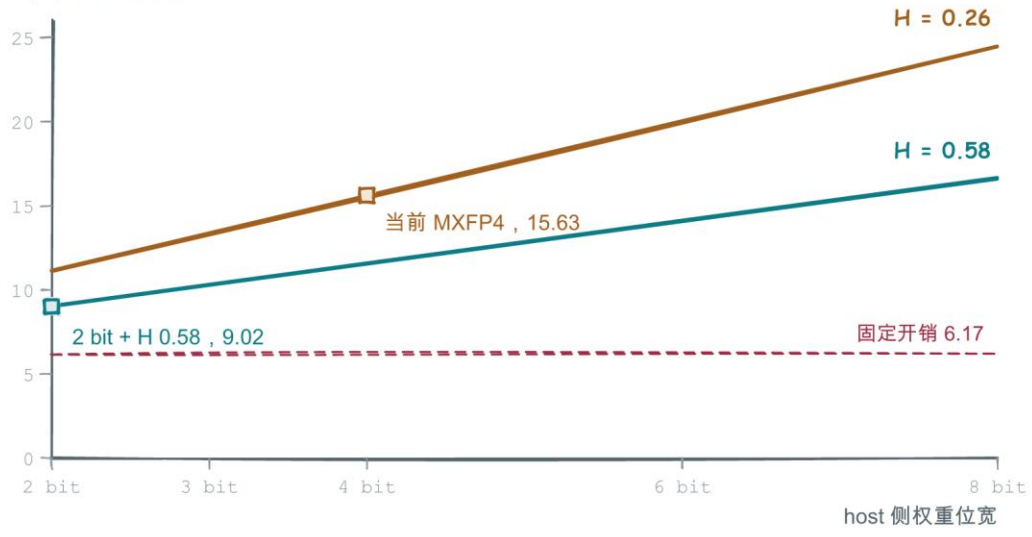
直线：CPU 要搬的字节正比于 (1 − H)，命中率每提 10 点省 1.28 ms

58% / 72% 是 prefill 后热专家置换的实测值，不含 decode 阶段预取；预取的目标是把 H 推得更高

host 侧位宽 vs「等 CPU」· 推演

每参数字节 = b/8 + 1/32（e8m0 scale）

纵轴：「等 CPU」ms/token



位宽与命中率相乘：4 → 2 bit 只省 47%，因为每 32 个值还带一个 e8m0 scale

decode 阶段的专家预取 提前一层预测并预取，或按带宽比例把部分 miss 经 H2D 交回 NPU；目标是把 H 推到 58% 之上

host 侧低比特 先用校准换精度余量，再用余量换位宽；按热度分级，冷专家用最小格式

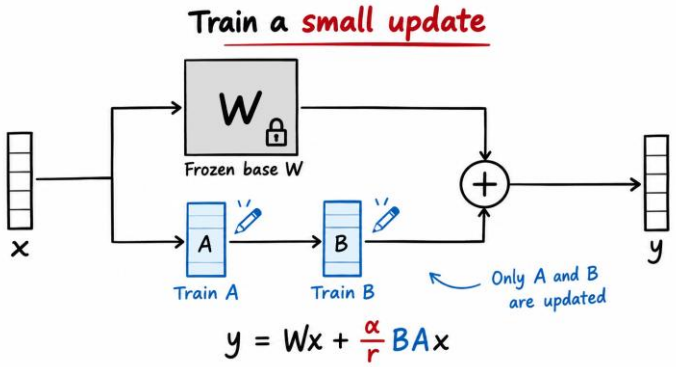
Engram 下沉 NVMe 表 203 GB 最冷，每 token 只读 48 行；主机侧权重内存约减半

固定开销 154.3 μs/层 只压 host 侧会把等待变成空闲，要让设备在等待期间有别的活干

CANN

单节点训练 · 基于 TorchTitan 单节点 DSV4-Flash LoRA 样例，64K 长序列 + rank64 支持

推理侧的约束是权重容量，训练侧还要叠加激活容量，后者随上下文长度增长。

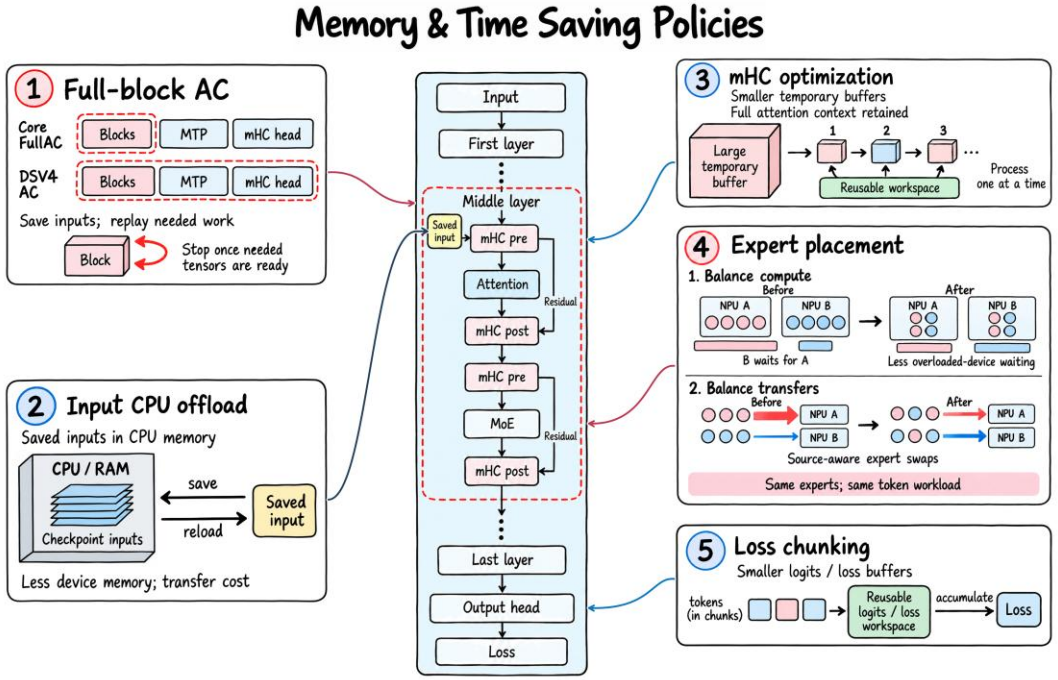


并行切分 CP 上下文并行 · EP 专家并行 · DPshard 参数分片

适配器 rank64 / alpha128，作用在 attention 与专家的投影上

片上内存优化 全块 AC、输入 CPU 卸载、mHC 优化、loss 分块

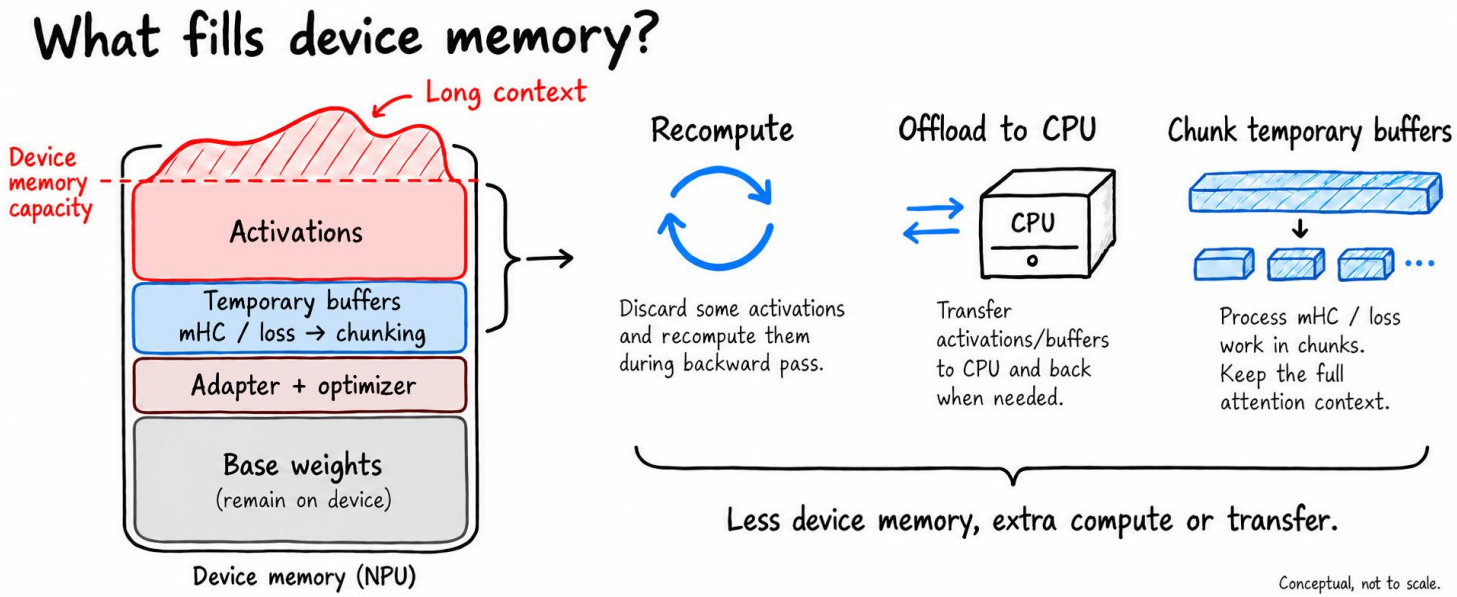
性能提升 专家重排



微调后推理侧：答案更简洁，生成耗时 -75.3%，精度基本持平

单节点训练 ·长上下文的内存管理 基于重算、卸载与分块的长序列省内存策略

LoRA 只省下可训练参数的梯度与优化器状态。64K 序列下占满片上内存的是激活、mHC 与 loss 的临时缓冲，以及分布式权重。



激活 全块 AC 重算，checkpoint 输入卸载到 CPU

临时缓冲 mHC 与 loss 按 token 分块，复用同一块工作区

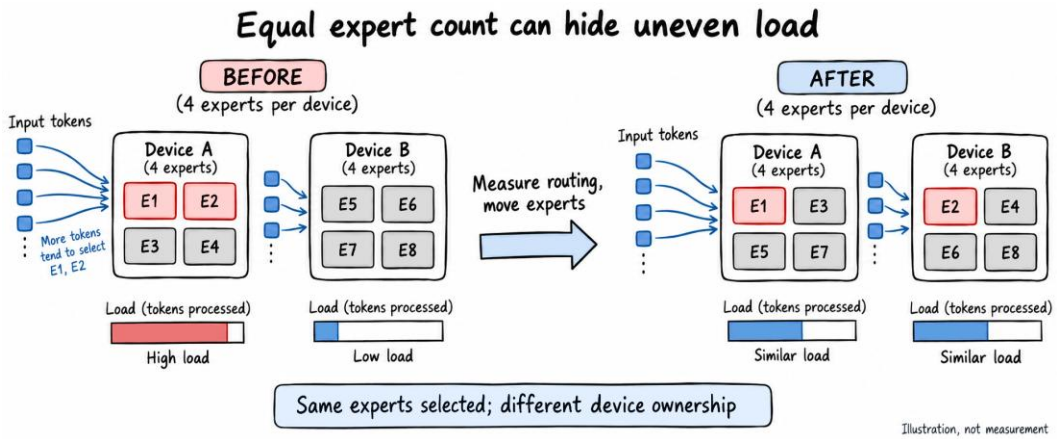
分布式权重 FSDP 前向后重新分片，下次用时再聚合

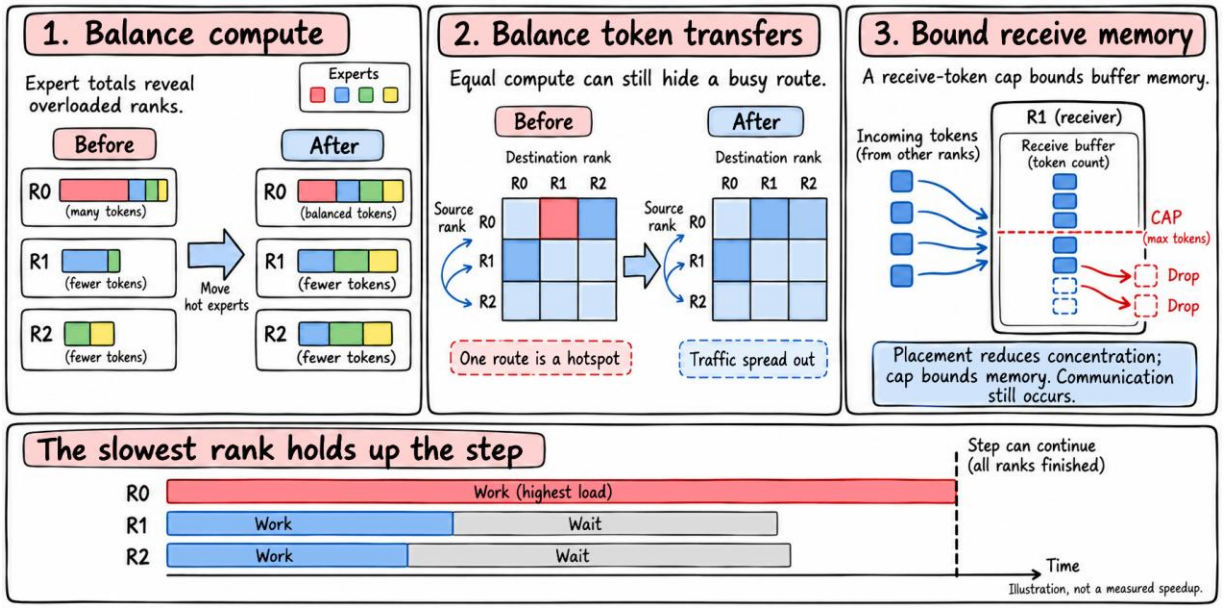
优化器状态 Adam 状态放 CPU

专家接收缓冲 定容接收池，超出容量的路由被丢弃

单节点训练 ·专家负载均衡 基于 token 分布标定的 expert balancing 算法，提升训练性能

MoE 每个 token 只选一部分专家，负载不均衡导致计算起泡。基于实测 token 分布决定专家落点，路由 bias 与每卡专家数都不变。





计算均衡 按每个专家的 token 总量调整各 rank 的工作量

传输均衡 按 源 rank → 目的 rank 的 token 数交换专家位置

接收上限 限制单卡收到的 token 数以约束缓冲内存

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态
社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem
Mission: Drive technological innovation through open source, lower development barriers, and jointly build a high-performance Ascend AI computing infrastructure.

上CANN社区获取干货

关注CANN公众号获取资讯

https://atomgit.com/cann

CANN